Estratégia Concursos

Aula 25

DataPrev (Analista de TI - Desenvolvimento de Software) Passo Estratégico de Conhecimentos Específicos - 2024 (Pós-Edital)

Autor:

09 de Setembro de 2024

11225354498 - Maria Vitória Gonçalves Vieira

Testes e Qualidade de Software

Sumário

Conteúdo 1
Glossário de termos 3
Roteiro de revisão 5
Qualidade de Software 5
Garantia x Controle de Qualidade 6
Verificação e Validação 8
Testes de Software 10
Abordagens de Testes 12
Níveis de Testes 16
Tipos de Testes 23
Aposta estratégica 28
Questões Estratégicas 29
Questionário de revisão e aperfeiçoamento 33
Perguntas 33
Perguntas e Respostas 34
Lista de Questões Estratégicas 36
Gabaritos 38

Conteúdo

Qualidade de Software. Controle de Qualidade. Garantia de Qualidade. Verificação e Validação. Testes de Software. Caixa Preta, Caixa Branca. Testes unitário, de integração, de validação e de sistema. Tipos variados de testes.

Análise Estatística

Inicialmente, convém destacar o percentual de incidência do assunto, dentro da disciplina **Engenharia de Software** em concursos/cargos similares. Quanto maior o percentual de cobrança de um dado assunto, maior sua importância.

Obs.: *um mesmo assunto pode ser classificado em mais de um tópico devido à multidisciplinaridade de conteúdo*.

11225354498 - Maria Vitória Gonçalves Vieira

Assunto	Relevância na disciplina em concursos similares
UML	**11.0 %**
Processos de Software - Desenvolvimento Ágil	**8.4 %**
Engenharia de Requisitos	**8.2 %**
Teste de Software	**6.7 %**
Métricas de Software	**5.1 %**
Desenvolvimento de Software	**4.6 %**
Inteligencia Artificial	**4.6 %**
Processos de Software	**4.6 %**
Orientação a Objetos	**4.5 %**
Gestão de Projetos em Engenharia de Software	**3.6 %**
Qualidade de Software	**3.4 %**
Metodologia de desenvolvimento de software	**2.5 %**
Gerência de Configuração	**1.9 %**
Geoprocessamento em Engenharia de Software	**1.6 %**
Prototipação	**1.2 %**
Conceitos Básicos em Engenharia de Software	**0.7 %**
Análise Estruturada	**0.6 %**
Ferramentas CASE	**0.4 %**
Ferramentas de Desenvolvimento de Software	**0.4 %**
Software livre	**0.3 %**
Manutenção de Software	**0.1 %**

11225354498 - Maria Vitória Gonçalves Vieira

Análise Essencial	**0.1 %**
Web 2.0	**0.1 %**
Portal Web	**0.1 %**
Refatoração	**0.1 %**
Engenharia da Informação	**0.1 %**

Glossário de termos

Faremos uma lista de termos que são relevantes ao entendimento do assunto desta aula. Caso tenha alguma dúvida durante a leitura, esta seção pode lhe ajudar a esclarecer.

Qualidade de software: É a medida de quão bem um software atende aos requisitos funcionais e não funcionais estabelecidos no processo de design e desenvolvimento. Inclui fatores como eficiência, confiabilidade, usabilidade e segurança.

Garantia de qualidade: Um processo proativo que se concentra em prevenir defeitos por meio do planejamento e da implementação de processos padronizados e eficazes. Ela se preocupa com a criação do ambiente e processos que produzirão um produto de alta qualidade.

Controle de qualidade: Uma atividade reativa que envolve a identificação e correção de defeitos no produto final. Isso geralmente envolve testes e inspeções para garantir que o produto atenda aos padrões de qualidade especificados.

Verificação: Verificação é o processo de avaliar produtos de trabalho de uma fase de desenvolvimento para determinar se atendem aos requisitos estabelecidos nessa fase.

Validação: A validação é o processo de avaliar um sistema durante ou no final do processo de desenvolvimento para determinar se ele atende aos requisitos do usuário.

Teste de software: Uma atividade de controle de qualidade que envolve a execução de um sistema de software para identificar e corrigir defeitos.

11225354498 - Maria Vitória Gonçalves Vieira

Teste caixa branca: Um método de teste de software que verifica a estrutura interna do código, focado no fluxo de controle dentro do software.

Teste caixa preta: Um método de teste de software que se concentra nos requisitos funcionais do software, sem considerar como o software funciona internamente.

Particionamento de equivalências: Uma técnica de teste de caixa preta que divide o input em classes de equivalência para reduzir o número de testes necessários.

Análise de valores limítrofes: Uma técnica de teste que se concentra nos limites ou extremos dos intervalos de entrada de um sistema.

Teste caixa cinza: Uma combinação de teste de caixa preta e caixa branca que considera a funcionalidade e a estrutura interna do software.

Teste unitário: Foca em verificar a menor unidade de código, como funções ou métodos.

Teste de integração: Verifica se diferentes unidades de código funcionam corretamente quando combinadas.

Teste de validação: Testa se o sistema como um todo atende aos requisitos do usuário final.

Teste alfa: Um tipo de teste de validação realizado em um ambiente controlado pela equipe de desenvolvimento.

Teste beta: Um tipo de teste de validação realizado em um ambiente real pelo usuário final.

Teste de Regressão: Verifica se as alterações no código não afetaram a funcionalidade existente do sistema.

Teste de Fumaça: Teste preliminar para verificar se as funções básicas do software estão funcionando antes de realizar testes mais detalhados.

Teste de Recuperação: Testa a capacidade do sistema de se recuperar de falhas ou erros.

Teste de Segurança: Verifica a capacidade do sistema de proteger seus dados e funcionar de forma estável sob condições de ameaça.

Teste de Desempenho: Avalia a velocidade, eficiência e escalabilidade do sistema sob diferentes cargas de trabalho.



11225354498 - Maria Vitória Gonçalves Vieira

Teste de Usabilidade: Avalia a experiência do usuário ao usar o sistema, incluindo a facilidade de uso, a intuitividade da interface e a satisfação do usuário.

ROTEIRO DE REVISÃO

A ideia desta seção é apresentar um roteiro para que você realize uma revisão completa do assunto e, ao mesmo tempo, destacar aspectos do conteúdo que merecem atenção.

Qualidade de Software

Qualidade de software se refere **à medida em que um sistema, componente ou processo de software atende a requisitos especificados e às necessidades ou expectativas do cliente ou usuário**. O termo também é usado para descrever o grau de excelência do software. É uma avaliação abrangente que envolve várias características, incluindo a funcionalidade do software, sua confiabilidade, usabilidade, eficiência, manutenibilidade e portabilidade.

A qualidade de software geralmente é avaliada por meio de "dimensões de qualidade", que são um conjunto de características utilizadas para avaliar a qualidade de um produto de software. Estas dimensões são baseadas em normas internacionais, como a ISO 25010, que sucedeu a ISO 9126. Essas normas propõem várias características de qualidade que podem ser medidas para avaliar a qualidade de um produto de software. As dimensões mais comuns são:

- **Funcionalidade:** Esta dimensão avalia se o software realiza as funções especificadas nos requisitos. Inclui a precisão, segurança e conformidade funcional do software.
- **Desempenho:** Avalia a performance do software sob certas condições. Inclui aspectos como tempo de resposta, utilização de recursos e capacidade de carga.
- **Compatibilidade:** Verifica se o software é capaz de interagir adequadamente com outros sistemas. Inclui a coexistência e a interoperabilidade.
- **Usabilidade:** Considera a facilidade de uso do software por parte dos usuários. Abrange alguns aspectos como a compreensibilidade, a aprendizagem, a operacionalidade e a atratividade do produto de software.
- **Confiabilidade:** Avalia a capacidade do software de manter o desempenho sob condições especificadas. Abrange a maturidade, a tolerância a falhas e a recuperabilidade do software.

11225354498 - Maria Vitória Gonçalves Vieira

- **Segurança:** Examina as capacidades do software de prevenir acessos não autorizados, manter a integridade dos dados e manter a confidencialidade das informações.
- **Manutenibilidade:** Analisa a facilidade com que o software pode ser corrigido, aprimorado e adaptado. Isso inclui a modularidade, a reusabilidade, a analisabilidade, a modificabilidade e a "testabilidade".
- **Portabilidade:** Esta dimensão considera a facilidade com que o software pode ser transferido de um ambiente para outro. Envolve a adaptabilidade, a "instalabilidade" e a substituição do ambiente.

Cada uma dessas dimensões oferece uma maneira de avaliar diferentes aspectos do software e ajuda a identificar áreas que precisam de melhoria e a garantir que o software atenda às necessidades e expectativas dos usuários.

Garantia x Controle de Qualidade

Garantia de Qualidade (Quality Assurance, QA) é uma atividade proativa que se concentra na prevenção de defeitos e na melhoria da qualidade dos processos usados para desenvolver e manter software. Seu principal objetivo é proporcionar visibilidade sobre o processo de desenvolvimento para permitir que a melhoria contínua seja possível. As atividades de garantia de qualidade envolvem a definição de processos e normas para desenvolvimento de software, auditoria e revisão de processos, treinamento e educação, e análise de métricas de processo.

Por exemplo, uma atividade de QA pode envolver a implementação de padrões de codificação e a realização de revisões de código para garantir que esses padrões sejam seguidos.

Controle de Qualidade (Quality Control, QC) é uma atividade reativa que se concentra na identificação e correção de defeitos em produtos de software. Seu principal objetivo é verificar a qualidade do produto de software, seja através de inspeções manuais ou por meio de testes automatizados.

Por exemplo, uma atividade de QC pode envolver a realização de testes de unidade, de integração e de sistema para identificar e corrigir defeitos no produto de software.

A principal diferença entre QA e QC é que a garantia de qualidade se concentra na prevenção de defeitos com o foco na melhoria dos processos utilizados para

11225354498 - Maria Vitória Gonçalves Vieira

desenvolver o produto, enquanto o controle de qualidade se concentra na identificação e correção de defeitos no próprio produto.

Em outras palavras, a garantia de qualidade tem mais a ver com a criação de um produto de alta qualidade, enquanto o controle de qualidade tem a ver com verificar que o produto resultante é de alta qualidade.

A tabela a seguir resume a diferença entre os dois:

	Garantia de Qualidade (QA)	Controle de Qualidade (QC)
Objetivo	Prevenir defeitos melhorando o processo.	Identificar e corrigir defeitos no produto.
Foco	Processos e sistemas.	Produtos e resultados.
Abordagem	Proativa - estabelece políticas e procedimentos para prevenir problemas.	Reativa - verifica o trabalho concluído para detectar problemas.
Atividades	Definição de processos, treinamento, auditorias de processo, revisões.	Testes de software, inspeções, revisões de código.
Exemplos	Implementação de padrões de codificação e revisão de código.	Testes de unidade, testes de integração, testes de sistema.
Responsabilidade	Normalmente é responsabilidade de todos os envolvidos no desenvolvimento do software.	Normalmente é uma equipe ou indivíduos designados, como engenheiros de teste.
Quando é realizado?	Durante todo o ciclo de vida do projeto, do início ao fim.	Geralmente é realizado após uma fase específica de produção (por exemplo, depois da codificação).

11225354498 - Maria Vitória Gonçalves Vieira

Verificação e Validação

Verificação e Validação são dois componentes fundamentais do processo de garantia da qualidade do software. Eles são comumente referidos juntos como V&V e são usados para garantir que o sistema atenda aos requisitos e especificações necessários.

Verificação:

É o processo que **verifica se o sistema está sendo construído corretamente, de acordo com as especificações e requisitos**. A verificação é uma atividade orientada a processos, o que significa que se concentra principalmente no processo de desenvolvimento de software e garantia de qualidade do software. As perguntas que a verificação busca responder são "Estamos construindo o sistema corretamente (do jeito certo)?" e "Estamos seguindo as especificações e requisitos definidos?"

Exemplos de atividades de verificação incluem revisões de projeto, inspeções de código, inspeções de documentos e análise estática de código.

Validação:

É o processo que **garante que o sistema atenda às necessidades e expectativas do usuário**. Diferentemente da verificação, a validação é uma atividade orientada a produtos. Isso significa que se concentra no produto de software final e se ele atende às necessidades do usuário final. As perguntas que a validação busca responder são "Estamos construindo o sistema certo?" e "O sistema atende às necessidades e requisitos do usuário?"

Exemplos de atividades de validação incluem testes de sistema, testes de aceitação do usuário e testes beta.

11225354498 - Maria Vitória Gonçalves Vieira





Por fim, as frases em inglês que definem V&V constituem um "trocadilho" interessante que pode lhe ajudar a lembrar dos conceitos na hora da prova. Veja:

Verificação: Are we building the product right?

Validação: Are we building the right product?

Testes de Software

Testes de Software são um conjunto de processos que têm como objetivo verificar e validar que um produto de software funciona corretamente e atende aos requisitos definidos. Os testes envolvem a execução do sistema para identificar qualquer gap, erro ou requisito faltante em contraste com os requisitos reais.

Testar o software é crucial para garantir a qualidade do produto final. Testes ajudam a identificar e corrigir bugs antes que o software seja lançado, garantindo que ele funcione conforme o esperado. Isso também contribui para a satisfação do usuário, confiabilidade do software, custos reduzidos a longo prazo (por meio da identificação precoce de problemas) e conformidade com padrões de qualidade e regulamentos.

De forma resumida, o principal objetivo de um teste é encontrar erros no produto de software. Porém, é importante notar que o processo de testes não pode garantir que o software está livre de erros, ele apenas mostra que erros e defeitos de software estão presentes.

Testes Manuais e Testes Automatizados

Testes Manuais são realizados por um humano que atua como usuário final e usa todas as funcionalidades do aplicativo para garantir o comportamento correto. Isso permite a identificação de problemas de usabilidade e experiência do usuário.

Testes Automatizados são realizados usando ferramentas de teste automatizadas para executar testes predefinidos em um aplicativo. Eles são úteis para tarefas repetitivas, como execução de regressão, e são eficientes em termos de tempo e custo para cenários de teste que precisam ser executados várias vezes.

A tabela abaixo faz uma análise comparativa entre testes manuais e automatizados:

	Teste Manual	Teste Automatizado
Definição	**Teste realizado por um ser humano que executa as ações do usuário.**	**Teste realizado por meio de scripts e ferramentas de automação.**
Custo inicial	**Baixo. Não requer ferramentas sofisticadas para começar.**	**Alto. Requer o desenvolvimento de scripts de teste e a compra de ferramentas de automação.**
Velocidade de execução	**Lenta. Depende da velocidade do testador humano.**	**Rápida. Uma vez criados, os scripts de teste podem ser executados**

11225354498 - Maria Vitória Gonçalves Vieira

		rapidamente e repetidamente.
Repetibilidade	**Baixa. O desempenho pode variar a cada teste, dependendo do testador.**	**Alta. Os scripts de teste podem ser executados exatamente da mesma maneira várias vezes.**
Escalabilidade	**Baixa. Aumentar o número de testes pode exigir mais testadores.**	**Alta. Os scripts de teste podem ser executados em várias máquinas simultaneamente.**
Detecção de erros	**Erros podem ser perdidos devido ao erro humano.**	**Consistente na detecção de erros, desde que o erro esteja dentro do escopo do script de teste.**
Exploração	**Excelente para testes exploratórios e identificação de problemas de usabilidade.**	**Não é adequado para testes exploratórios. Focado em cenários de teste predefinidos.**
Manutenção	**Baixa. As mudanças no software podem exigir apenas pequenas alterações nos procedimentos de teste.**	**Alta. As mudanças no software podem exigir a atualização de scripts de teste.**

Ciclo de Vida de Testes

O ciclo de vida dos testes de software consiste em várias fases que a equipe de testes segue para garantir a qualidade do produto final. Embora possa haver diferentes modelos, um ciclo de vida de teste típico inclui:

1. Planejamento de Testes: Nesta fase, os testadores definem a estratégia e planejam os testes, incluindo a escolha das ferramentas de teste e a definição dos recursos necessários.

11225354498 - Maria Vitória Gonçalves Vieira

2. **Análise de Testes:** Aqui, a equipe de testes analisa os requisitos do software e identifica o que precisa ser testado.
3. **Design de Testes:** Os testadores projetam os casos de teste e identificam os dados de teste necessários.
4. **Implementação de Testes:** Casos de teste são implementados e preparados para o ambiente de teste.
5. **Execução de Testes:** Os casos de teste são executados e os resultados são documentados.
6. **Relatório e Acompanhamento de Defeitos:** Qualquer defeito encontrado é relatado e rastreado até a resolução.
7. **Reteste e Regressão:** Após a correção dos defeitos, os testes são repetidos para garantir que os problemas foram resolvidos e que nenhuma nova falha foi introduzida.
8. **Encerramento de Testes:** Uma vez que todos os defeitos foram corrigidos e os casos de teste passaram, o processo de teste é concluído.

Abordagens de Testes

As abordagens de testes se dividem em Testes Caixa-Branca, Testes Caixa-Preta e Testes Caixa-Cinza. A primeira foca nas estruturas internas dos procedimentos do sistema. A segunda foca nas entradas e saídas especificadas nos requisitos funcionais. Por fim, a terceira foca tanto em estruturas internas quanto nas entradas e saídas especificadas nos requisitos. Vejamos em detalhes cada uma delas.

Abordagem Caixa Branca

Testes caixa branca, também conhecidos como testes de caixa transparente ou teste estrutural, envolvem a **análise do código interno e a estrutura de um programa**. Este tipo de teste é realizado por desenvolvedores que **têm conhecimento sobre o código interno do sistema**. O propósito principal do teste de caixa branca é melhorar a segurança, a estrutura e a funcionalidade do código ao detectar erros e bugs ocultos.

Estratégias de Teste Caixa Branca:

- **Cobertura de Caminho:** Verifica todos os caminhos possíveis do programa. Por exemplo, cada declaração e ramificação é coberta.

11225354498 - Maria Vitória Gonçalves Vieira

- **Cobertura de Declaração:** Garante que cada linha ou instrução do código seja executada pelo menos uma vez.
- **Cobertura de Condição:** Verifica todas as condições (por exemplo, em instruções if, for, while) para ambas as saídas possíveis: verdadeiro e falso.
- **Cobertura de Ramificação ou Decisão:** Garante que cada ponto de decisão seja executado tanto para verdadeiro quanto para falso.

Considere o seguinte exemplo de código em Python:

```python
def soma_positiva(a, b):
    if a > 0 and b > 0:
        return a + b
    else:
        return "Os números devem ser positivos"
```

Usando a estratégia de **cobertura de caminho**, você deve testar todos os caminhos possíveis, ou seja, os casos quando a e b são positivos e quando não são.

Usando a estratégia de **cobertura de declaração**, você deve garantir que cada linha do código seja executada pelo menos uma vez. Para este exemplo, isso significaria testar o método com valores positivos e valores não positivos.

Usando a **estratégia de cobertura de condição**, você deve garantir que todas as condições sejam avaliadas como verdadeiras e falsas. Aqui, você deve garantir que a condição a > 0 and b > 0 seja verdadeira e falsa.

Usando a **estratégia de cobertura de ramificação**, você deve garantir que cada ponto de decisão seja executado tanto para verdadeiro quanto para falso. Para este exemplo, isso significaria testar quando a > 0 and b > 0 é verdadeiro e quando é falso.

Abordagem Caixa Preta

Testes caixa preta são uma forma de testar em que o funcionamento interno do item a ser testado **não é considerado pelo testador**. Em vez disso, o testador está **apenas interessado em saber se o sistema produz a saída correta em resposta à entrada dada, sem se preocupar com como o sistema chegou a essa saída**. O principal propósito dos testes caixa preta é validar o comportamento funcional do software, garantindo que ele esteja de acordo com as especificações e requisitos do usuário.

11225354498 - Maria Vitória Gonçalves Vieira

Estratégias de Teste Caixa Preta:

- Particionamento de Equivalência: Esta é uma técnica de teste de software que divide o domínio de entrada do sistema em classes de equivalência, das quais os testes são derivados. A ideia é que, se o software funciona corretamente para um valor em uma classe de equivalência, ele funcionará corretamente para todos os outros valores na mesma classe.
- Análise de Valores Limítrofes: Esta é uma técnica de teste que se concentra nos valores limite ou extremos do domínio de entrada. Isso é feito porque muitos erros ocorrem nos limites de condições de entrada.

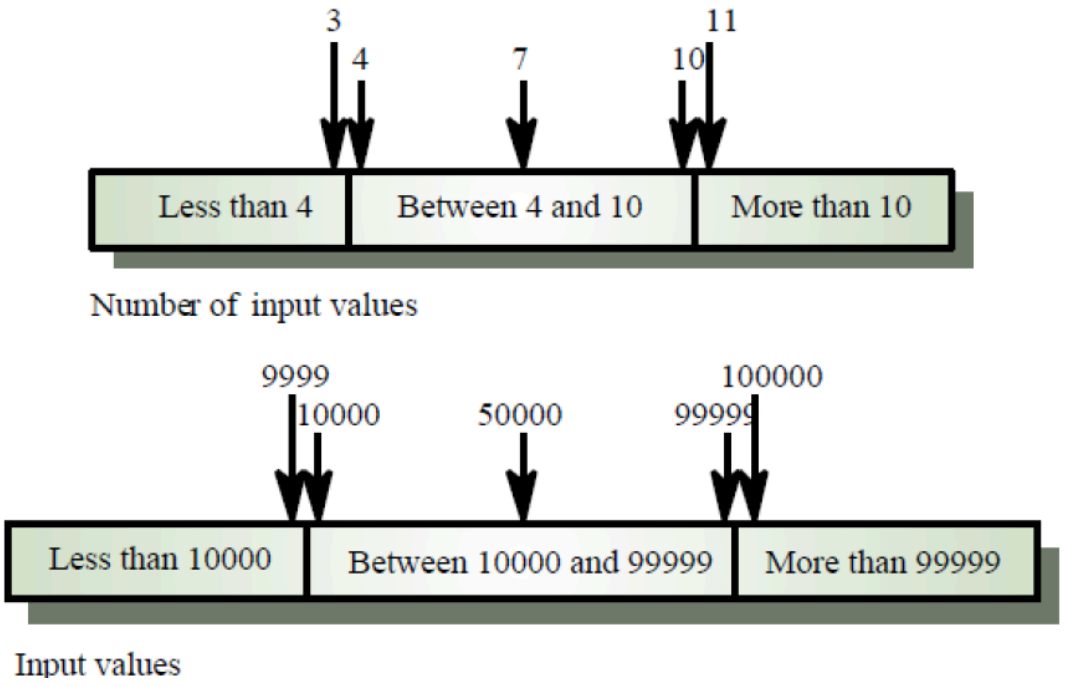
Por exemplo, considere uma função que recebe uma idade e retorna se a pessoa é maior de idade ou não. Neste caso, o domínio de entrada são todos os números inteiros positivos.

Usando a estratégia de particionamento de equivalência, você dividiria o domínio de entrada em duas classes: idades menores que 18 e idades maiores ou iguais a 18. Testar pelo menos um valor de cada classe seria suficiente para garantir a corretude do software para todas as idades.

Usando a estratégia de análise de valores limítrofes, você testaria a função com idades como 17 e 18, que são os valores de "limite" (ou seja, valores próximo do mínimo ou máximo definidos). Dado que muitos erros ocorrem nos limites das condições de entrada, este teste pode ajudar a identificar esses possíveis erros.

A figura a seguir mostra o particionamento de equivalência e as entradas limites (mínimas e máximas) de uma função hipotética:

11225354498 - Maria Vitória Gonçalves Vieira



Abordagem Caixa Cinza

Os testes de caixa cinza são uma combinação de testes de caixa preta e caixa branca. Nesta abordagem, o testador tem conhecimento parcial do sistema interno. Ele geralmente tem acesso a um projeto detalhado do sistema, representando o conhecimento da estrutura interna, mas ainda assim, o teste é realizado a partir de uma perspectiva de caixa preta.

Esse método é especialmente útil quando se trata de testes de integração de sistemas maiores, onde o testador precisa entender como as partes individuais funcionam juntas, mas não necessariamente os detalhes internos de cada componente.

Benefícios desta abordagem incluem:

- Maior Eficiência de Testes: O testador pode se concentrar nos aspectos mais importantes do sistema sem se preocupar com todos os detalhes internos.
- Melhor Cobertura de Testes: Com algum conhecimento da estrutura interna, o testador pode projetar testes mais eficazes para cobrir casos que poderiam ser ignorados em uma abordagem puramente caixa preta.
- Detecção de Erros Mais Rápida: Com um conhecimento parcial do sistema interno, o testador pode ser capaz de identificar e isolar problemas mais rapidamente do que com uma abordagem puramente caixa preta.

Por exemplo, em um aplicativo web, um testador de caixa cinza pode ter acesso ao código HTML da página web e ao banco de dados subjacente. O testador pode usar essa informação para realizar testes mais eficazes, como tentar inserir tipos específicos de dados no banco de dados através do site para testar a segurança e a validação de dados. Ao mesmo tempo, o testador ainda estará testando o site a partir da perspectiva de um usuário final, garantindo que a funcionalidade e a experiência do usuário atendam às expectativas.

Níveis de Testes

De maneira geral, podemos dizer que o teste começa pelo "pequeno" e passa para o "grande". Em outras palavras, os testes iniciais focalizam um único componente ou um pequeno grupo de componentes relacionados e aplicam-se testes para descobrir erros nos dados e na lógica de processamento que foram encapsulados pelo(s) componente(s). Depois de testados, os componentes devem ser integrados até que o sistema completo esteja pronto.

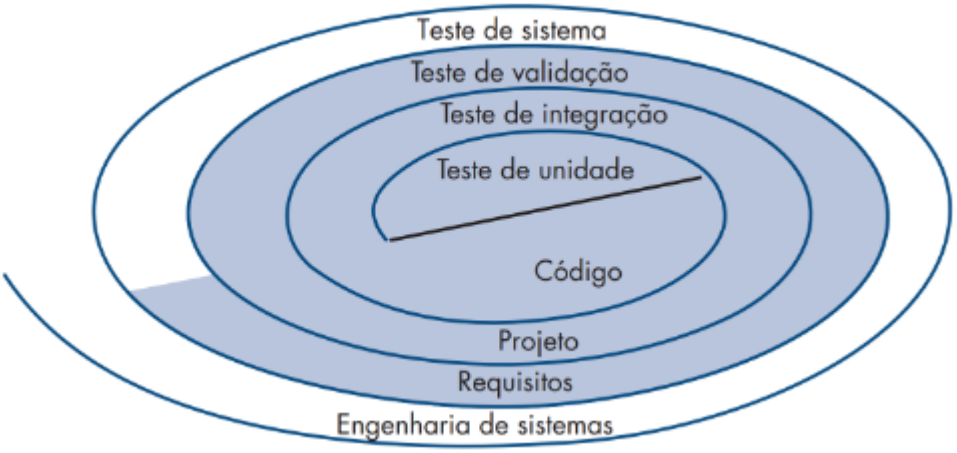
Assim, uma estratégia de teste de software deve acomodar testes de baixo nível, necessários para verificar se um pequeno segmento de código fonte foi implementado corretamente, bem como testes de alto nível, que validam as funções principais do sistema de acordo com os requisitos do cliente.

A literatura costuma dividir os níveis de testes em:

- Teste unitário, relacionado ao código (baixo nível) do sistema;
- Teste de integração, relacionado ao projeto arquitetural do sistema;
- Teste de validação (aceitação), mais próximo do usuário; e
- Teste de sistema, relacionado à engenharia de sistemas.

A figura abaixo do Pressman ilustra bem isso:

11225354498 - Maria Vitória Gonçalves Vieira



Teste Unitário

Teste unitário é um nível de teste de software onde os componentes individuais de um software são testados. A finalidade é validar que cada unidade de software funciona como foi projetada. Uma unidade é a menor parte testável do software. Geralmente tem um ou poucos inputs e geralmente um único output. Em programação orientada a objetos, uma unidade é frequentemente um método de uma classe.

O propósito do teste unitário é isolar uma seção do código e verificar sua corretude. Um teste unitário busca garantir que o código produz a saída esperada para uma dada entrada. Essa garantia pode ajudar os desenvolvedores a modificar e melhorar o código sem medo de quebrar funcionalidades existentes, facilitando a manutenção e expansão do software.

Testes unitários são frequentemente automatizados, mas também podem ser realizados manualmente. Para escrever um teste unitário, você precisa conhecer a lógica do código para criar entradas que possam testar completamente o código para uma ampla variedade de situações.

Por exemplo, se você tiver uma função que some dois números, um teste unitário para essa função verificaria coisas como o que acontece quando você soma dois números positivos, um número positivo e um negativo, dois negativos, um número e zero, e assim por diante. Se a função é projetada para lidar apenas com números inteiros, o teste unitário também verificaria o que acontece quando você tenta usar números flutuantes ou outros tipos de dados.

Veja algumas definições que já caíram em prova sobre teste de unidade:

11225354498 - Maria Vitória Gonçalves Vieira

DEFINIÇÕES de prova - Testes de unidade
Testes de Unidade são aqueles realizados sobre as menores estruturas de código-fonte, como métodos e classes.
Testes de Unidade consistem em testar individualmente, componentes ou módulos de *software* que, posteriormente devem ser testados de maneira integrada.
Testes de Unidade focalizam cada componente de um software de forma individual, garantindo que o componente funciona adequadamente.
Testes de Unidade focalizam o esforço de verificação na menor unidade de projeto de software, isto é, no componente ou no módulo de software.
Testes de Unidade têm por objetivo explorar a menor unidade do projeto, procurando identificar falhas ocasionadas por defeitos de lógica e de implementação em cada módulo separadamente.
Testes de Unidade enfocam a lógica interna de processamento e as estruturas de dados dentro dos limites de um componente.
Testes de Unidade concentram o esforço de verificação na menor unidade de design de software.
Testes de Unidade concentram-se na lógica de processamento interno e nas estruturas de dados dentro dos limites de um componente.
Testes de Unidade têm por objetivo explorar a menor unidade do projeto, procurando provocar falhas ocasionadas por defeitos de lógica e de implementação em cada módulo, separadamente.
Testes de Unidade têm como foco as menores unidades de um programa, que podem ser funções, procedimentos, métodos ou classes.

Teste de Integração

O teste de integração é um nível de teste de software em que módulos individuais são combinados e testados em grupo. É realizado após os testes unitários, e antes dos testes de sistema.

O objetivo do teste de integração é detectar falhas nas interações entre módulos integrados. À medida que os módulos são desenvolvidos e testados individualmente, eles precisam ser combinados e testados juntos para garantir que funcionem corretamente como um grupo.

Técnicas de Teste de Integração:

- Big-Bang: Todos os módulos são integrados de uma só vez e depois testados. É uma abordagem eficaz quando o projeto é pequeno e os módulos estão fortemente relacionados.
- Top-Down: O teste começa pelo módulo de nível superior e gradualmente desce para os módulos de nível inferior. Se um módulo de nível inferior ainda não está pronto, um "stub" (esqueleto) pode ser usado em seu lugar.
- Bottom-Up: O teste começa pelos módulos de nível inferior e gradualmente sobe para os módulos de nível superior. Se um módulo de nível superior ainda não está pronto, um "driver" (algo que simula a execução) pode ser usado em seu lugar.
- Sanduíche (ou Híbrido): Uma combinação de abordagens de top-down e bottom-up. Os módulos de nível médio são testados usando uma abordagem de top-down e bottom-up.

Por exemplo, suponha que você esteja construindo um software de carrinho de compras online. Existem vários módulos, incluindo usuário, catálogo de produtos, carrinho de compras, pagamento, etc.

No teste de integração Big-Bang, todos esses módulos seriam integrados e testados de uma vez. Você testaria a funcionalidade do carrinho de compras como um todo depois que todos os módulos fossem integrados.

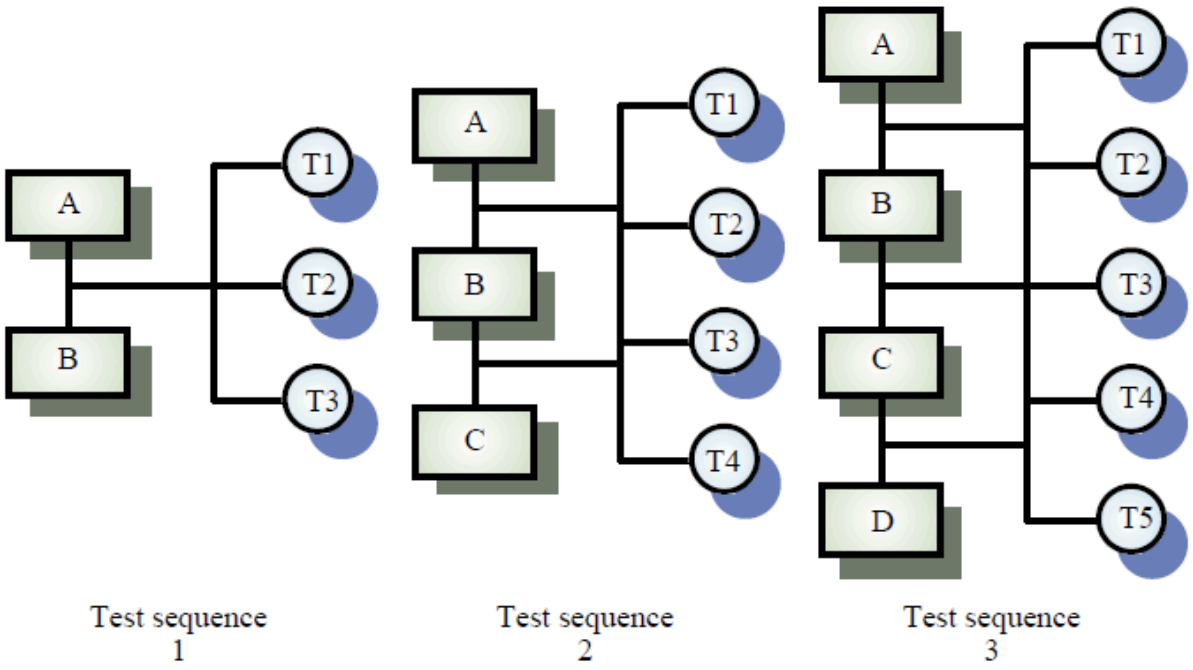
Na abordagem Top-Down, você pode começar com o módulo do usuário e, em seguida, integrar e testar o módulo do catálogo de produtos, seguido pelos módulos do carrinho de compras e do pagamento.

No teste Bottom-Up, você pode começar com o módulo de pagamento e, em seguida, subir para o módulo do carrinho de compras, catálogo de produtos e usuário.

Na abordagem Sanduíche, você começaria com o módulo do usuário (topo) e o módulo de pagamento (fundo), e então trabalharia em direção ao meio (catálogo de produtos e carrinho de compras).

Veja um exemplo em que testes de integração são executados de forma gradual:

11225354498 - Maria Vitória Gonçalves Vieira



Teste de Validação (Aceitação)

O teste de validação é o processo de avaliar o software durante ou no final do processo de desenvolvimento para determinar se ele atende aos requisitos especificados.

O propósito dos testes de validação é demonstrar que o sistema cumpre a intenção para a qual foi criado. Isso envolve a verificação de todas as funcionalidades do sistema e a validação de acordo com as necessidades do usuário final. Em outras palavras, certifica-se de que "o produto certo foi construído".

Processo de Validação:

1. O primeiro passo na validação é estabelecer os requisitos que o sistema deve cumprir. Isso geralmente é feito durante a fase de análise de requisitos do processo de desenvolvimento de software.
2. Depois que os requisitos são estabelecidos, os testes de validação são planejados. O plano de teste de validação incluirá detalhes sobre quais aspectos do sistema serão testados, como eles serão testados e os critérios de aceitação que devem ser atendidos.
3. A execução dos testes de validação então ocorre. Isso pode envolver testar o sistema como um todo ou testar funcionalidades individuais do sistema.

4. Os resultados dos testes de validação são registrados e comparados com os critérios de aceitação estabelecidos. Se os resultados dos testes atenderem aos critérios de aceitação, a validação é considerada bem-sucedida. Se os critérios de aceitação não forem atendidos, serão necessárias mudanças e melhorias no sistema.

A literatura destaca dois tipos principais de testes de aceitação: Teste Alfa e Teste Beta.

Teste Alfa:

O teste alfa é um dos tipos de teste de validação que é realizado no ambiente de desenvolvimento antes de o produto ser entregue ao cliente. O objetivo desse teste é encontrar todos os possíveis problemas e erros antes de o produto chegar ao usuário final.

Durante o teste alfa, os usuários reais do software trabalham com o software em um ambiente "controlado", supervisionados por engenheiros que registram problemas e feedback. Esse processo ajuda a identificar quaisquer problemas que possam ocorrer no mundo real e dá aos desenvolvedores a oportunidade de corrigir esses problemas antes de o software ser lançado.

Teste Beta:

O teste beta é realizado por um grupo de usuários finais em um ambiente de "produção". Neste estágio, o software é considerado 90%-95% completo e os desenvolvedores buscam feedback do usuário final sobre a funcionalidade, usabilidade, confiabilidade e compatibilidade do produto.

Os usuários beta são normalmente clientes ou usuários finais potenciais do software. Eles usam o software da mesma forma que usariam em condições normais de trabalho, e qualquer problema que encontrem é reportado de volta para os desenvolvedores. Isso permite que os desenvolvedores corrijam quaisquer problemas que possam ter sido perdidos durante os testes anteriores antes de o software ser lançado oficialmente.

Veja a tabela abaixo que resume as principais diferenças entre Teste Alfa e Teste Beta:

	Teste Alfa	Teste Beta
Local de Execução	**Realizado no ambiente de desenvolvimento da empresa**	**Realizado no ambiente do usuário final**

11225354498 - Maria Vitória Gonçalves Vieira

Quem Executa	**Normalmente membros da equipe de desenvolvimento ou QA, por vezes com o envolvimento de usuários internos**	**Usuários finais reais do software**
Foco	**Identificar bugs e problemas antes da liberação para testes beta**	**Verificar a usabilidade, funcionalidade, confiabilidade e compatibilidade do software no mundo real**
Feedback	**Direto, pois é supervisionado pelos desenvolvedores ou testadores da equipe**	**Indireto, coletado através de pesquisas, questionários, entrevistas e relatórios de erros gerados pelo usuário**
Controle	**Alto controle, pois é conduzido em um ambiente controlado**	**Baixo controle, pois é conduzido no ambiente do usuário sem supervisão direta**
Objetivo	**Validar se o software funciona como esperado, identificar bugs e problemas**	**Obter feedback do usuário real para melhorias e garantir que o software atenda às necessidades do usuário final**

Teste de Sistema

Software é apenas um elemento de um grande sistema de computador. Ao final do desenvolvimento, ele precisará ser incorporado aos outros elementos do sistema (por exemplo: hardware, pessoas, informações). Um problema clássico de teste de sistema é a "procura do culpado". Isso ocorre quando um erro é descoberto e os desenvolvedores de diversos elementos do sistema começam a acusar um ao outro pelo problema.

11225354498 - Maria Vitória Gonçalves Vieira

Em vez de adotar essa postura sem sentido, você deve se antecipar aos problemas potenciais de interface e realizar as seguintes atividades:

- Criar caminhos de manipulação de erro que testem todas as informações vindas de outros elementos do sistema;
- Executar uma série de testes que simulem dados incorretos ou outros erros potenciais na interface de software;
- Deve registrar os resultados dos testes para utilizar como evidência se ocorrer a caça ao culpado;

- Participar do planejamento e projeto de testes do sistema para assegurar que o software seja testado adequadamente.

O Teste de Sistema é na realidade uma série de diferentes testes cuja **finalidade primária é exercitar totalmente o sistema**. Embora cada um dos testes tenha uma finalidade diferente, todos funcionam no sentido de verificar se os elementos do sistema foram integrados adequadamente e executam as funções a eles alocadas

Em resumo, o teste de sistema é a garantia final de que o software irá funcionar em seu ambiente "maior" ou "real", considerando não apenas o próprio sistema, mas hardware, pessoas, processos e outras informações ao seu redor.

Tipos de Testes

Agora veremos vários tipos de testes. É importante que você saiba que cada autor pode criar seu tipo de teste e eles – por vezes – se contradizem. A ideia aqui é apresentar aqueles testes que possuem maior destaque entre os autores mais consagrados de engenharia de software, mas deixando claro que não se trata de uma lista exaustiva. Eventualmente, você pode encontrar um teste que não foi mencionado em aula, pois a lista é imensa. Vamos, então, ver os tipos de testes mais comuns e mais cobrados em prova.

Teste de Regressão

O teste de regressão é um tipo de teste realizado para garantir que as mudanças recentes no código não afetem o comportamento existente do software. Isso inclui novas funcionalidades, melhorias de código, correções de bugs, etc.

O objetivo do teste de regressão é garantir que o software ainda funciona corretamente após as mudanças. Toda vez que o código é modificado, há uma chance de que uma parte existente e funcional do software possa ser inadvertidamente afetada. Os testes de regressão ajudam a identificar tais problemas antes que o software seja lançado.

Estratégias de Teste de Regressão:

- Teste de Regressão Completa: Envolve a reexecução de todos os testes existentes no conjunto de testes. Esta é a abordagem mais segura, mas também a mais demorada.
- Teste de Regressão Seletiva: Envolve a reexecução de um subconjunto de testes existentes que podem ser afetados pela mudança. A seleção de testes é baseada em uma análise de que partes do software podem ser afetadas pela mudança.
- Teste de Regressão Priorizado: Semelhante ao teste de regressão seletiva, mas os testes são priorizados com base em critérios como áreas críticas de negócios, funcionalidades frequentemente usadas, áreas de código com alto risco, etc.

Por exemplo, suponha que você tenha um site de comércio eletrônico e a equipe de desenvolvimento fez algumas alterações no módulo de pagamento para suportar uma nova forma de pagamento. Testes de regressão que podem ser realizados:

1. Verifique se as formas de pagamento existentes ainda estão funcionando corretamente.
2. Verifique se a adição de itens ao carrinho de compras e a visualização do carrinho ainda estão funcionando corretamente.
3. Verifique se os recursos de login e logout do usuário ainda estão funcionando corretamente.

Cada um desses testes tem o objetivo de verificar se os recursos existentes ainda estão funcionando corretamente após a mudança no módulo de pagamento.

Teste de Fumaça

Testes de fumaça, também conhecidos como "build verification tests", consistem em um conjunto de testes que são realizados em uma versão recém-criada de um software, antes de realizarem testes mais rigorosos.

O objetivo do teste de fumaça é verificar se as funcionalidades básicas de um software funcionam corretamente. Eles são projetados para serem rápidos e devem tocar todas as partes da aplicação de maneira superficial. Se um teste de fumaça falhar, a build é rejeitada e a equipe de desenvolvimento é notificada para corrigir o problema antes que quaisquer outros testes sejam realizados.

Quando aplicar o teste de fumaça:

- Após uma nova build ou versão do software ser criada: Para garantir que a build é estável e que as funcionalidades básicas estão funcionando como esperado.
- Antes de outros testes detalhados serem realizados: Para economizar tempo e recursos, evitando testes mais rigorosos em uma build instável.

Por exemplo, suponha que você esteja trabalhando em um aplicativo de compras online. Exemplos de testes de fumaça que você poderia realizar:

1. Verificar se o aplicativo é iniciado corretamente.
2. Verificar se os usuários podem fazer login e logout com sucesso.
3. Verificar se os usuários podem adicionar itens ao carrinho de compras.
4. Verificar se os usuários podem realizar uma busca de produtos.
5. Verificar se os usuários podem finalizar uma compra.

Se qualquer um desses testes falhar, a build seria considerada instável e o problema precisaria ser corrigido antes de prosseguir com testes mais detalhados.

Teste de Recuperação

O teste de recuperação é um tipo de teste que verifica como um sistema se recupera de falhas, erros ou qualquer outra forma de interrupção. Ele valida que o software pode retornar com sucesso ao seu estado normal após um crash ou falha.

O objetivo do teste de recuperação é assegurar que o sistema pode lidar com falhas e ainda recuperar sem perda de dados ou funcionalidade. Isso é especialmente importante para sistemas que lidam com informações críticas, onde a perda de dados pode ter consequências graves. Além disso, esses testes ajudam a identificar e corrigir problemas que podem causar falhas no sistema.

Por exemplo, suponha que você tenha um banco de dados de um e-commerce. Testes de recuperação que você poderia realizar:

11225354498 - Maria Vitória Gonçalves Vieira

1. Recuperação de falhas de energia: Desligue a energia inesperadamente e, em seguida, reinicie o sistema para verificar se todos os dados estão intactos e se o sistema retorna ao seu estado normal.
2. Recuperação de falhas no sistema: Simule uma falha no sistema, como um crash no servidor, e observe como o sistema se recupera. Verifique se todos os dados estão intactos e se o sistema consegue se recuperar e continuar a funcionar como esperado.
3. Recuperação de falhas na rede: Desconecte a rede ou simule uma falha na rede. Verifique se o sistema pode lidar com essa falha e se recupera quando a conexão de rede é restaurada.

Em cada um desses casos, o objetivo é verificar se o sistema pode se recuperar de uma falha, sem perda de dados ou funcionalidade.

Teste de Segurança

Testes de segurança são uma série de métodos e práticas projetadas para identificar e corrigir vulnerabilidades de segurança em um sistema de software. Esses testes são essenciais para garantir que as informações e os sistemas sejam protegidos contra ameaças.

O principal objetivo dos testes de segurança é identificar todas as possíveis vulnerabilidades ou fraquezas no sistema de software e garantir que os dados e os recursos do sistema estejam protegidos contra possíveis ataques. Ele visa prevenir ameaças, como acesso não autorizado, perda de dados, interrupções no serviço e outros.

Veja alguns tipos de ameaças à segurança da informação que podem ser testadas para identificar se o sistema está sujeito a elas:

1. Injeção de SQL: Esta é uma técnica onde o invasor injeta código SQL malicioso que pode levar à perda de dados ou ao roubo de informações sensíveis.
2. Cross-Site Scripting (XSS): Este tipo de ataque ocorre quando os invasores injetam scripts maliciosos em sites confiáveis.
3. Ataques de Força Bruta: Este tipo de ataque ocorre quando um invasor tenta obter acesso a um sistema através da tentativa e erro de várias combinações de senhas.
4. Ataques DDoS: Um ataque DDoS sobrecarrega o sistema com tráfego, tornando-o inacessível para usuários legítimos. Testes de segurança devem verificar se o sistema pode lidar com altos volumes de tráfego e se possui medidas de proteção adequadas contra tais ataques.

11225354498 - Maria Vitória Gonçalves Vieira

Teste de Desempenho

O teste de desempenho é um tipo de teste que é realizado para avaliar a velocidade, a capacidade de resposta e a estabilidade de um sistema de software. Na verdade, ele abrange vários "subtipos" de testes, permitindo medir e avaliar a qualidade dos atributos de desempenho do sistema, como tempo de resposta, confiabilidade, uso de recursos e escalabilidade.

O propósito principal do teste de desempenho é garantir que o sistema de software atenda aos critérios de desempenho estabelecidos durante a fase de design. Ele ajuda a identificar e corrigir problemas de desempenho, como atrasos no processamento ou na resposta, gargalos e sobrecarga do sistema. O teste de desempenho é fundamental para garantir a satisfação do usuário, especialmente para aplicações web e móveis, onde a alta capacidade de resposta e a rapidez são essenciais para uma boa experiência do usuário.

Técnicas de Teste de Desempenho:

1. Teste de Carga: Este teste verifica o comportamento do sistema sob uma carga específica, geralmente aumentando a carga de maneira progressiva. Por exemplo, você poderia simular a situação de milhares de usuários acessando o site ao mesmo tempo, colocando itens no carrinho e efetuando check-out.
2. Teste de Estresse: Este teste verifica a estabilidade e confiabilidade do sistema sob condições extremas. Ele é projetado para testar o sistema além de seus limites normais para identificar como o sistema falha e como ele se recupera. Por exemplo, você poderia simular picos extremos de tráfego no site, como durante uma venda de Black Friday ou Cyber Monday, para ver se o sistema consegue lidar com a carga.
3. Teste de Estabilidade Este teste é uma variante do teste de carga onde o sistema é submetido a carga durante um período prolongado para verificar se ele pode sustentar uma carga contínua. Por exemplo, você poderia executar uma carga constante no site durante um dia inteiro ou mais para ver se surgem problemas de desempenho ao longo do tempo.
4. Teste de Volume: Este teste verifica o comportamento do sistema quando submetido a uma grande quantidade de dados. Por exemplo, você poderia testar a plataforma com um número extremamente alto de produtos, usuários e transações para verificar se ela consegue lidar com grandes volumes de dados.

Em todos os casos, o objetivo é identificar e corrigir problemas de desempenho para garantir que o sistema possa fornecer uma experiência de usuário rápida e eficiente, mesmo sob condições de alta demanda.

11225354498 - Maria Vitória Gonçalves Vieira

Teste de Usabilidade

O teste de usabilidade é uma técnica de teste usada para **avaliar um produto ou sistema pela perspectiva do usuário final**. Ele se concentra em medir a facilidade de uso de um produto, quão intuitivo ele é e quanto tempo leva para o usuário realizar tarefas específicas.

O principal objetivo do teste de usabilidade é garantir que o produto ou sistema é fácil de usar e que fornece uma boa experiência ao usuário. Ele ajuda a identificar áreas problemáticas, confusões e melhorias potenciais, a partir da perspectiva do usuário. Além disso, visa a garantir que o produto ou sistema atenda às necessidades e expectativas do usuário.

Por exemplo, suponha que você esteja testando a usabilidade de um aplicativo de compras online. Alguns exemplos de tarefas que você poderia incluir em seus cenários de teste incluem:

1. Navegar pelo catálogo de produtos e encontrar um item específico.
2. Adicionar um item ao carrinho de compras e proceder para o checkout.
3. Procurar informações sobre políticas de devolução e garantia.
4. Enviar uma pergunta ou solicitação através da seção de contato ou suporte do aplicativo.

Em cada caso, você observaria quanto tempo leva para o participante completar a tarefa, quão fácil ou difícil eles acham a tarefa e se eles encontram algum erro ou problema. Com base nos resultados, você poderia então fazer ajustes para melhorar a usabilidade do aplicativo.

APOSTA ESTRATÉGICA

A ideia desta seção é apresentar os pontos do conteúdo que mais possuem chances de serem cobrados em prova, considerando o histórico de questões da banca em provas de nível semelhante à nossa, bem como as inovações no conteúdo, na legislação e nos entendimentos doutrinários e jurisprudenciais[1].

[1] Vale deixar claro que nem sempre será possível realizar uma aposta estratégica para um determinado assunto, considerando que às vezes não é viável identificar os pontos mais prováveis de serem cobrados a partir de critérios objetivos ou minimamente razoáveis.

Os testes de caixa preta são uma técnica de teste de software na qual o foco está na entrada e saída do sistema sem considerar a estrutura interna ou a implementação do código subjacente. Esta abordagem trata o software como uma caixa preta, onde o testador desconhece os detalhes internos. O principal objetivo é verificar a funcionalidade do software, garantindo que ele atenda às especificações do usuário e funcione conforme esperado sob variadas condições de uso. Os testes de caixa preta são amplamente utilizados para validar a adequação do software em termos de comportamento e conformidade com os requisitos, com técnicas comuns incluindo o particionamento de equivalência e a análise de valores limítrofes.

Em contraste, os testes de caixa branca, também conhecidos como testes de caixa transparente ou testes estruturais, exigem um conhecimento profundo da estrutura interna do código do software. Os testadores que executam testes de caixa branca utilizam esse conhecimento para examinar os fluxos de controle dentro do aplicativo, incluindo caminhos, loops e condições. Essa abordagem permite que os testadores verifiquem a eficácia dos caminhos de execução, a cobertura do código e a identificação de possíveis pontos fracos no software. O teste de caixa branca é essencial para garantir a qualidade interna do software, permitindo uma identificação mais direcionada e a correção de bugs em um nível mais granular.

Questões Estratégicas

Nesta seção, apresentamos e comentamos uma amostra de questões objetivas selecionadas estrategicamente: são questões com nível de dificuldade semelhante ao que você deve esperar para a sua prova e que, em conjunto, abordam os principais pontos do assunto.

A ideia, aqui, não é que você fixe o conteúdo por meio de uma bateria extensa de questões, mas que você faça uma boa revisão global do assunto a partir de, relativamente, poucas questões

1. (FGV / TCE-TO – 2022) O analista de sistemas Carlos está desenvolvendo o software CharlieApp e implementou o teste C. O teste C consiste apenas em determinar se o método A do código de CharlieApp retorna o resultado esperado C ao chamar o método B que realiza uma consulta ao banco de dados de CharlieApp.

   Portanto, o teste C implementado por Carlos é de:

   a) unidade;

11225354498 - Maria Vitória Gonçalves Vieira

b) aceitação;

c) ponta a ponta;

d) exploração;

e) integração.

Comentários:

Note que temos dois métodos: A e B. O Teste C consiste em verificar se o Método A, ao chamar o Método B, retorna o resultado esperado. Logo, o intuito é examinar se ambos os métodos estão bem integrados, logo é um teste de integração.

Gabarito: Letra E

2. (FGV / IMBEL - 2021) Com referência às metodologias de teste de software, a técnica que avalia as funcionalidades sem ter contato com o código-fonte, mas apenas com as respostas que o sistema dá a determinadas ações, é conhecida como:

a) Caixa Branca.

b) Caixa Cinza.

c) Caixa Preta.

d) Regressão.

e) Testes não funcionais.

Comentários:

(a) Errado, o teste caixa branca tem contato com o código-fonte; (b) Errado, esse teste permitiria o acesso a partes do código-fonte; (c) Correto, avaliar funcionalidades sem ter contato com o código-fonte é função da técnica caixa preta; (d) Errado, teste de regressão não é uma técnica e, sim, um tipo de teste; (e) Errado, testes não funcionais não são técnicas de software (aliás, desconheço testes não-funcionais).

11225354498 - Maria Vitória Gonçalves Vieira

Gabarito: Letra C

3. (FGV / FUNSAÚDE-CE - 2021) No contexto da testagem de software, os testes do tipo Unitário, aplicam-se normalmente:

a) à aderência a padrões.

b) às funções codificadas.

c) às interfaces de entrada de dados.

d) à integração dos componentes.

e) aos limites de carga.

Comentários:

(a) Errado, isso seria função da análise estática de código-fonte; (b) Correto, o teste unitário é bastante focado em funções/procedimentos do código-fonte; (c) Errado, isso seria função do teste caixa-preta; (d) Errado, isso seria função do teste de integração; (e) Errado, isso seria função do teste de carga.

Gabarito: Letra B

4. (FGV / TCE-AM - 2021) A Equipe de Desenvolvimento de Software (EDS) de um tribunal de contas está trabalhando na construção de componentes de um novo sistema de software. Para verificar o funcionamento do software no nível de componente, a EDS deverá aplicar testes de caixa:

a) branca, para validar parâmetros de entrada;

b) preta, para garantir que caminhos independentes dos componentes tenham sido testados;

c) branca do tipo análise de valor-limite;

d) preta como alternativa a testes de caixa branca;

e) branca, para exercitar decisões lógicas em seus lados verdadeiro e falso.

Comentários:

11225354498 - Maria Vitória Gonçalves Vieira

Já que se deseja verificar o funcionamento do software no nível de componentes, entende-se que não se trata de suas interfaces e, sim, de seu funcionamento interno. Logo, trata-se de um teste caixa-branca. Vamos analisar as alternativas: (a) Errado, isso seria uma prática do teste caixa-preta; (b) Errado, o teste no nível de componente é um teste caixa-branca; (c) Errado, análise do valor-limite é um teste caixa-preta; (d) Errado, o teste no nível de componente é um teste caixa-branca e funciona de maneira bastante díspar dos testes caixa-preta; (e) Correto, testes caixa-branca realmente buscam exercitar decisões lógicas internas em seus lados verdadeiros e falsos.

Gabarito: Letra E

5. (FGV / DPE-RJ – 2019) Uma empresa foi contratada por um órgão governamental para modificar e adaptar um sistema para gerenciamento eletrônico de documentos, com base nas especificações criadas pelo próprio órgão. A contratada entregou ao órgão uma parte do sistema com as alterações solicitadas, e um grupo de usuários finais do sistema está simulando operações de rotina, para atestar se seu comportamento está de acordo com as expectativas da empresa.

Conclui-se que está sendo realizado o teste de:

a) unidade;

b) regressão;

c) integração;

d) aceitação;

e) cobertura.

Comentários:

Vejam que um grupo de usuários finais estão testando o sistema, portanto, trata-se do teste de aceitação, que verifica se o software está de acordo com os requisitos estabelecidos.

Gabarito: Letra D

11225354498 - Maria Vitória Gonçalves Vieira

QUESTIONÁRIO DE REVISÃO E APERFEIÇOAMENTO

A ideia do questionário é elevar o nível da sua compreensão no assunto e, ao mesmo tempo, proporcionar uma outra forma de revisão de pontos importantes do conteúdo, a partir de perguntas que exigem respostas subjetivas.

São questões um pouco mais desafiadoras, porque a redação de seu enunciado não ajuda na sua resolução, como ocorre nas clássicas questões objetivas.

O objetivo é que você realize uma auto explicação mental de alguns pontos do conteúdo, para consolidar melhor o que aprendeu ;)

Além disso, as questões objetivas, em regra, abordam pontos isolados de um dado assunto. Assim, ao resolver várias questões objetivas, o candidato acaba memorizando pontos isolados do conteúdo, mas muitas vezes acaba não entendendo como esses pontos se conectam.

Assim, no questionário, buscaremos trazer também situações que ajudem você a conectar melhor os diversos pontos do conteúdo, na medida do possível.

É importante frisar que não estamos adentrando em um nível de profundidade maior que o exigido na sua prova, mas apenas permitindo que você compreenda melhor o assunto de modo a facilitar a resolução de questões objetivas típicas de concursos, ok?

Nosso compromisso é proporcionar a você uma revisão de alto nível!

Vamos ao nosso questionário:

Perguntas

1. O que é qualidade de software?
2. Qual é a diferença entre garantia de qualidade e controle de qualidade?
3. O que é verificação no contexto de testes de software?
4. Como a validação difere da verificação?
5. O que é teste caixa branca?
6. Como o teste caixa preta difere do teste caixa branca?

11225354498 - Maria Vitória Gonçalves Vieira

7. O que é particionamento de equivalências?
8. O que é análise de valores limítrofes?
9. O que é teste unitário?
10. O que é teste de integração?
11. Qual é o propósito do teste de validação?
12. O que distingue o teste alfa do teste beta?
13. O que é teste de regressão?
14. Qual é o objetivo do teste de fumaça?
15. O que é teste de recuperação?
16. O que é teste de segurança?
17. O que é teste de desempenho?
18. Quais são algumas das técnicas de teste de desempenho?
19. O que é teste de usabilidade?
20. O que é um teste caixa cinza?

Perguntas e Respostas

1. O que é qualidade de software?
   Resposta: A qualidade de software é a medida de quão bem um software atende aos requisitos funcionais e não funcionais estabelecidos no processo de design e desenvolvimento. Inclui fatores como eficiência, confiabilidade, usabilidade e segurança.
2. Qual é a diferença entre garantia de qualidade e controle de qualidade?
   Resposta: A garantia de qualidade é um processo proativo que se concentra em prevenir defeitos por meio do planejamento e da implementação de processos padronizados e eficazes. O controle de qualidade, por outro lado, é uma atividade reativa que envolve a identificação e correção de defeitos no produto final.
3. O que é verificação no contexto de testes de software?
   Resposta: Verificação é o processo de avaliar produtos de trabalho de uma fase de desenvolvimento para determinar se atendem aos requisitos estabelecidos nessa fase.
4. Como a validação difere da verificação?
   Resposta: A validação é o processo de avaliar um sistema durante ou no final do processo de desenvolvimento para determinar se ele atende aos requisitos do usuário. Portanto, a validação se preocupa com "Estamos construindo o produto certo?" e a verificação com "Estamos construindo o produto corretamente?".

11225354498 - Maria Vitória Gonçalves Vieira

5. O que é teste caixa branca?
   Resposta: Teste caixa branca é um método de teste de software que verifica a estrutura interna do código, focado no fluxo de controle dentro do software.
6. Como o teste caixa preta difere do teste caixa branca?
   Resposta: O teste caixa preta se concentra nos requisitos funcionais do software, sem considerar como o software funciona internamente. Difere do teste caixa branca que examina a estrutura interna do código.
7. O que é particionamento de equivalências?
   Resposta: Particionamento de equivalências é uma técnica de teste de caixa preta que divide o input em classes de equivalência para reduzir o número de testes necessários.
8. O que é análise de valores limítrofes?
   Resposta: Análise de valores limítrofes é uma técnica de teste que se concentra nos limites ou extremos dos intervalos de entrada de um sistema.
9. O que é teste unitário?
   Resposta: Teste unitário é um método de teste que foca em verificar a menor unidade de código, como funções ou métodos.
10. O que é teste de integração?
    Resposta: Teste de integração é uma estratégia de teste que verifica se diferentes unidades de código funcionam corretamente quando combinadas.
11. Qual é o propósito do teste de validação?
    Resposta: O teste de validação tem como objetivo testar se o sistema como um todo atende aos requisitos do usuário final.
12. O que distingue o teste alfa do teste beta?
    Resposta: O teste alfa é um tipo de teste de validação realizado em um ambiente controlado pela equipe de desenvolvimento, enquanto o teste beta é realizado em um ambiente real pelo usuário final.
13. O que é teste de regressão?
    Resposta: O teste de regressão verifica se as alterações no código não afetaram a funcionalidade existente do sistema.
14. Qual é o objetivo do teste de fumaça?
    Resposta: O teste de fumaça é um teste preliminar para verificar se as funções básicas do software estão funcionando antes de realizar testes mais detalhados.

15. O que é teste de recuperação?
    Resposta: O teste de recuperação testa a capacidade do sistema de se recuperar de falhas ou erros.
16. O que é teste de segurança?
    Resposta: O teste de segurança verifica a capacidade do sistema de proteger seus dados e funcionar de forma estável sob condições de ameaça.

11225354498 - Maria Vitória Gonçalves Vieira

17. O que é teste de desempenho?
    Resposta: O teste de desempenho avalia a velocidade, eficiência e escalabilidade do sistema sob diferentes cargas de trabalho.
18. Quais são algumas das técnicas de teste de desempenho?
    Resposta: As técnicas de teste de desempenho incluem testes de carga, estresse, estabilidade e volume.
19. O que é teste de usabilidade?
    Resposta: O teste de usabilidade avalia a experiência do usuário ao usar o sistema, incluindo a facilidade de uso, a intuitividade da interface e a satisfação do usuário.
20. O que é um teste caixa cinza?
    Resposta: O teste caixa cinza é uma combinação de teste de caixa preta e caixa branca que considera tanto a funcionalidade quanto a estrutura interna do software.

Lista de Questões Estratégicas

1. (FGV / TCE-TO – 2022) O analista de sistemas Carlos está desenvolvendo o software CharlieApp e implementou o teste C. O teste C consiste apenas em determinar se o método A do código de CharlieApp retorna o resultado esperado C ao chamar o método B que realiza uma consulta ao banco de dados de CharlieApp.

   Portanto, o teste C implementado por Carlos é de:

   a) unidade;

   b) aceitação;

   c) ponta a ponta;

   d) exploração;

   e) integração.

2. (FGV / IMBEL - 2021) Com referência às metodologias de teste de software, a técnica que avalia as funcionalidades sem ter contato com o código-fonte, mas apenas com as respostas que o sistema dá a determinadas ações, é conhecida como:

a) Caixa Branca.

b) Caixa Cinza.

c) Caixa Preta.

d) Regressão.

e) Testes não funcionais.

3. (FGV / FUNSAÚDE-CE - 2021) No contexto da testagem de software, os testes do tipo Unitário, aplicam-se normalmente:

a) à aderência a padrões.

b) às funções codificadas.

c) às interfaces de entrada de dados.

d) à integração dos componentes.

e) aos limites de carga.

4. (FGV / TCE-AM - 2021) A Equipe de Desenvolvimento de Software (EDS) de um tribunal de contas está trabalhando na construção de componentes de um novo sistema de software. Para verificar o funcionamento do software no nível de componente, a EDS deverá aplicar testes de caixa:

a) branca, para validar parâmetros de entrada;

b) preta, para garantir que caminhos independentes dos componentes tenham sido testados;

c) branca do tipo análise de valor-limite;

d) preta como alternativa a testes de caixa branca;

e) branca, para exercitar decisões lógicas em seus lados verdadeiro e falso.

5. (FGV / DPE-RJ – 2019) Uma empresa foi contratada por um órgão governamental para modificar e adaptar um sistema para gerenciamento

eletrônico de documentos, com base nas especificações criadas pelo próprio órgão. A contratada entregou ao órgão uma parte do sistema com as alterações solicitadas, e um grupo de usuários finais do sistema está simulando operações de rotina, para atestar se seu comportamento está de acordo com as expectativas da empresa.

Conclui-se que está sendo realizado o teste de:

a) unidade;

b) regressão;

c) integração;

d) aceitação;

e) cobertura.

Gabaritos

1. E
2. C
3. B
4. E
5. D

11225354498 - Maria Vitória Gonçalves Vieira

ESSA LEI TODO MUNDO CONHECE:
PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.

1. 

Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.

2. 

Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).

3. 

Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).

4. 

Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).

5. 

Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).

6. 

Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).

7. 

Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.

8. 

O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.